

手摸手之乘车人如何实现分库分表

分库分表介绍

乘车人的分库分表相关理论参考《用户数据如何分库分表文章》。

[用户数据如何实现分库分表](#)

乘车人分库分表策略

乘车人的数据严重依赖于用户数据。每个用户至少需要有一个对应的乘车人，即自己本人。当然，也有可能是其他人，因为允许用户注册账号后为他人购票的情况。这种关联确保了用户和乘车人之间的正确映射，使系统能够准确地处理购票和相关信息。

根据上述前提，让我们进行一些分析，来看看哪些因素会影响乘车人数据量：

1. 首先，每个用户至少会有一个对应的乘车人信息。因此，乘车人数据量至少等于用户数据量。
2. 对于情侣购票用户，有两种情况：一种是普通场景下只有一人购票，另一种是在极端情况下，双方都添加乘车人信息，以便更方便地抢票。
3. 家庭购票用户可能会在一次购票中为全家人购买车票。不过，也有可能其他家庭成员并没有注册 12306 账号。
4. 职场购票用户中，可能会有一人代表多名员工出差，购买所有人员的车票。

当然，上述因素并不能穷举所有情况。因此，在系统设计时，我们综合考虑各种情况得出一个经验值，即乘车人数据量约为用户数据量的 4 倍。虽然这个估算可能不是完全准确，但我们希望能在容量规划时考虑到一定的宽裕余量。

对于分库分表容量评估，我们通常会尽可能地全面的评估。这样做的好处是，即使每张表的数据量不大，也能及早发现拆分后是否存在数据问题，以便及时进行调整和优化。

需要特别指出的是，我们对表数据量的考虑阈值相对较小，这是因为我们的系统具备良好的可扩展性，能够轻松应对大量的数据增长。因此，基于这样的分库分表策略，即使在几百年后，这个分库分表仍能处理数据且不会出现性能问题。这为我们的系统提供了稳定可靠的性能保障。

选择乘车人分片键

选择分库分表中的分片键（Sharding Key）是一个关键决策，它直接影响了分库分表的性能和可扩展性。以下是一些选择分片键的关键因素：

1. 访问频率：选择分片键应考虑数据的访问频率。将经常访问的数据放在同一个分片上，可以提高查询性能和降低跨分片查询的开销。
2. 数据均匀性：分片键应该保证数据的均匀分布在各个分片上，避免出现热点数据集中在某个分片上的情况。
3. 业务关联性：分片键应该与业务关联紧密，这样可以避免跨分片查询和跨库事务的复杂性。
4. 数据不可变：一旦选择了分片键，它应该是不可变的，不能随着业务的变化而频繁修改。

乘车人数据的查询必然会涉及到用户信息，而我们的用户表是按照 username 进行分片的。鉴于这些因素，我们决定将 username 作为乘车人表的分片键。

改造乘车人分库分表

如果你没有使用过 ShardingSphere 分库分表操作，可以查看官网进行一些前置条件理解。

数据分片 :: [ShardingSphere](#)

引入 ShardingSphere 依赖

```
<dependency>
  <groupId>org.apache.shardingsphere</groupId>
  <artifactId>shardingsphere-jdbc-core</artifactId>
  <version>5.3.2</version>
</dependency>
```

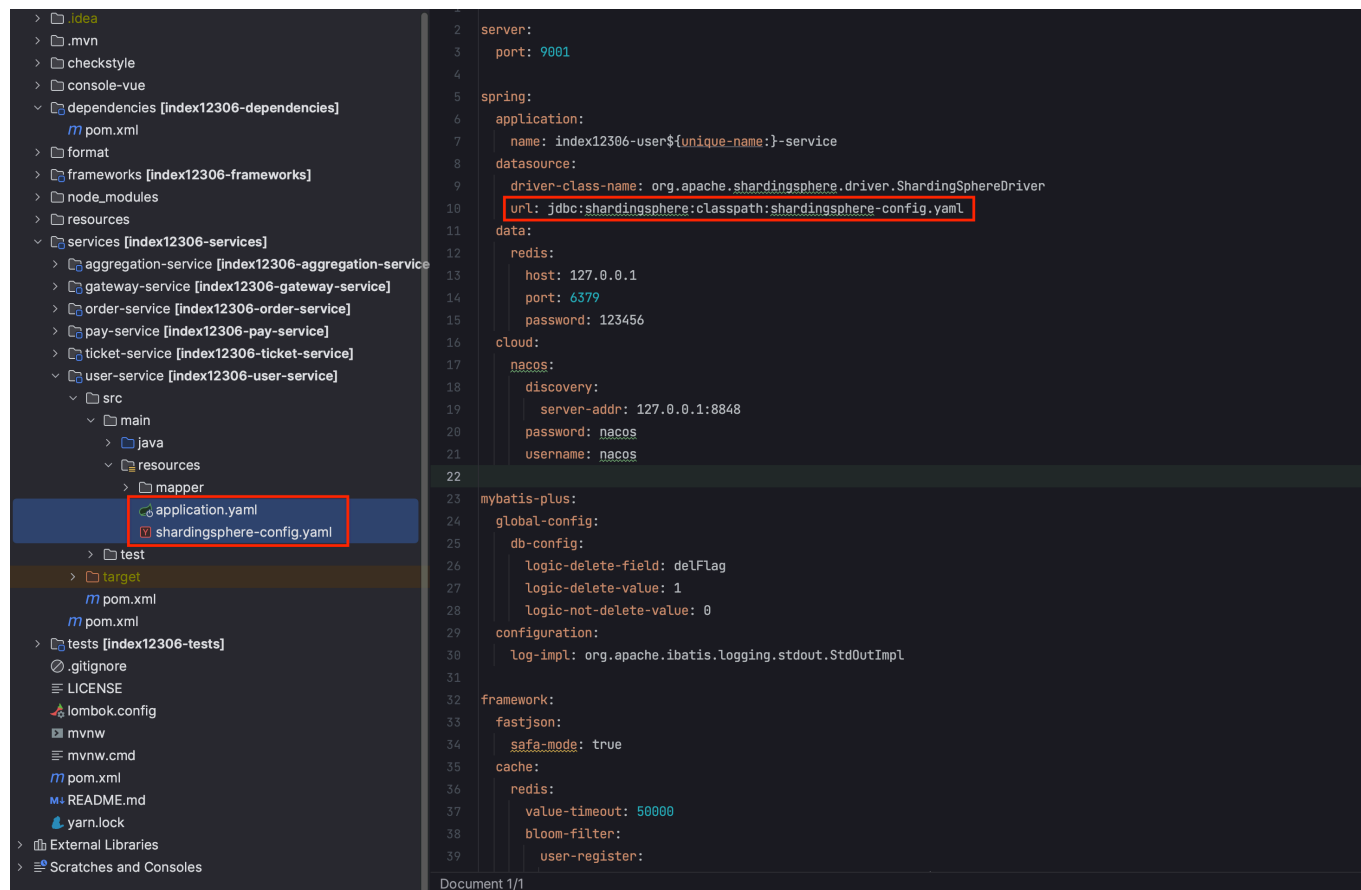
定义分片规则

用户和乘车人都在 user-service 服务中，所以两者共用一个配置文件。

```
spring:
  datasource:
    # ShardingSphere 对 Driver 自定义，实现分库分表等隐藏逻辑
    driver-class-name: org.apache.shardingsphere.driver.ShardingSphereDriver
```

```
# ShardingSphere 配置文件路径
url: jdbc:shardingsphere:classpath:shardingsphere-config.yaml
```

配置文件地址信息如下：



乘车人分片配置

因为 12306 更多的是向大家演示分库分表，所以分 2 个库以及对应业务 16 张表。

shardingsphere-config.yaml

```
# 数据源集合
datasources:
  ds_0:
    dataSourceClassName: com.zaxxer.hikari.HikariDataSource
    driverClassName: com.mysql.cj.jdbc.Driver
    jdbcUrl: jdbc:mysql://127.0.0.1:3306/12306_user_0?
    useUnicode=true&characterEncoding=UTF-
    8&rewriteBatchedStatements=true&allowMultiQueries=true&serverTimezone=Asia/Shanghai
    username: root
```

```

password: root

ds_1:
  dataSourceClassName: com.zaxxer.hikari.HikariDataSource
  driverClassName: com.mysql.cj.jdbc.Driver
  jdbcUrl: jdbc:mysql://127.0.0.1:3306/12306_user_1?
  useUnicode=true&characterEncoding=UTF-
  8&rewriteBatchedStatements=true&allowMultiQueries=true&serverTimezone=Asia/Shanghai
  username: root
  password: root

rules:
- !SHARDING
  tables:
    t_passenger:
      # 真实数据节点，比如数据库源以及数据库在数据库中真实存在的
      actualDataNodes: ds_${0..1}.t_user_${0..15}
      # 分库策略
      databaseStrategy:
        # 用于单分片键的标准分片场景
        standard:
          # 分片键
          shardingColumn: username
          # 分片算法，对应 rules[0].shardingAlgorithms
          shardingAlgorithmName: passenger_database_hash_mod
      # 分表策略
      tableStrategy:
        # 用于单分片键的标准分片场景
        standard:
          # 分片键
          shardingColumn: username
          # 分片算法，对应 rules[0].shardingAlgorithms
          shardingAlgorithmName: passenger_table_hash_mod
      # 分片算法
      shardingAlgorithms:
        # 数据库分片算法
        passenger_database_hash_mod:
          # 根据分片键 Hash 分片
          type: HASH_MOD
          # 分片数量
          props:

```

```
        sharding-count: 2
# 数据表分片算法
passenger_table_hash_mod:
    # 根据分片键 Hash 分片
    type: HASH_MOD
    # 分片数量
    props:
        sharding-count: 16
# 展现逻辑 SQL & 真实 SQL
props:
    sql-show: true
```

写好以上配置后，就可以愉快使用数据库分库分表了。

`props.sql-show` 配置尽量打开，它能打印应用的逻辑 SQL 以及真正查询数据库的真实 SQL。

文末总结

分库分表是一种数据库架构设计和优化手段，用于应对大规模数据量和高并发访问的情况。它通过将原本的单库单表拆分为多个数据库或表，来分散数据存储和查询压力，从而提高系统的性能和可扩展性。一般情况下，引入分库分表是为了解决系统运行过程中出现的数据量庞大或查询性能慢等问题。

分库分表主要有两种模式：垂直拆分和水平拆分。垂直拆分是将原本的单库拆分为多个数据库，每个数据库存储特定的业务数据，如用户库、订单库等。水平拆分是将原本的单表拆分为多个表，每个表存储部分数据，如订单表拆分为 `OrderTable\_0`、`OrderTable\_1` 等。

在选择分库分表时，关键因素包括数据的访问频率、数据均匀性、业务关联性以及分片键的不可变性。分片键是用于决定数据分片的字段，合理选择分片键可以提高查询性能和降低跨分片查询的开销。对于用户数据分库分表的场景，我们根据当前用户表的数据量和每年新增用户数，预估了未来的数据量。选择了适当的分片键，并进行了分库分表的配置，以保证系统在未来较长时间内的性能和可扩展性。

我们选择了使用 ShardingSphere 来实现分库分表，因为 ShardingSphere 拥有活跃的社区、高质量的代码和丰富的功能，能够满足多样化的业务需求。根据实际情况，我们可以选择 ShardingSphere 的 JDBC 或 Proxy 模式来进行分库分表操作，以适应项目的具体需求。

综上所述，分库分表是一种重要的数据库优化策略，可以帮助解决大规模数据和高并发访问带来的性能问题。在设计分库分表方案时，需要考虑系统的实际情况和未来的数据增长，合理选择分片键和分片策略，以保证系统的稳定性和可扩展性。同时，选择一个可靠的分库分表工具，如 ShardingSphere，可以帮助我们更加便捷地实现分库分表操作。

更新: 2023-09-06 18:24:49

原文: <<https://www.yuque.com/magestack/12306/zhsauz6ksng8wvgf>>